

Com base nos princípios de **Clean Architecture** e **SOLID**, a estrutura para o agendamento de vacinas e exames no seu zoológico deve ser isolada do mecanismo de persistência, focando nas regras de negócio e na separação de responsabilidades 1-3. Como você já possui o cadastro de animais, a nova entidade de agendamento se relacionará com eles através de um identificador único 4. Aqui está a arquitetura proposta, limitada às entidades, relações e cenários de agendamento e listagem: **### 1. Camada de Domínio (Domain)** As **Entidades** são o núcleo da aplicação e contêm as regras de negócio de alto nível 5, 6. Para o agendamento, utilizaremos uma classe que valida se a data é válida e define o tipo de procedimento (vacina ou exame) 7, 8. typescript //

```
src/domain/entities/medical-appointment.ts import { randomUUID } from "crypto"; export type AppointmentType = 'VACCINE' | 'EXAM'; export interface MedicalAppointmentProps { animalId: string; description: string; type: AppointmentType; date: Date; } export class MedicalAppointment { private props: MedicalAppointmentProps; private _id: string; constructor(props: MedicalAppointmentProps, id?: string) { // Regra de Negócio: Não permite agendamentos no passado [9] if (props.date < new Date()) { throw new Error("A data do agendamento não pode ser no passado."); } this.props = props; this._id = id ?? randomUUID(); } get id() { return this._id; } get animalId() { return this.props.animalId; } get type() { return this.props.type; } get date() { return this.props.date; } get description() { return this.props.description; } } ### 2. Camada de Aplicação (Application) Esta camada define os
```

**Casos de Uso** (ações do sistema) e os **Contratos de Repositório** (interfaces) 10, 11. **#### Interface do Repositório** O repositório serve para a comunicação com o banco de dados sem expor detalhes técnicos (como SQL ou ORMs) para a regra de negócio 10, 12.

```
typescript // src/application/repositories/IMedicalAppointmentRepository.ts import { MedicalAppointment } from "../../domain/entities/medical-appointment"; export interface IMedicalAppointmentRepository { create(appointment: MedicalAppointment): Promise; findAllByAnimal(animalId: string): Promise; listAll(): Promise; } #### Casos de Uso (SOLID: Responsabilidade Única) Cada classe executa apenas uma tarefa específica 2, 13.
```

```
typescript // src/application/use-cases/schedule-appointment-use-case.ts import { MedicalAppointment, AppointmentType } from "../../domain/entities/medical-appointment"; import { IMedicalAppointmentRepository } from "../../repositories/IMedicalAppointmentRepository"; interface ScheduleRequestDTO { animalId: string; description: string; type: AppointmentType; date: Date; } export class ScheduleAppointmentUseCase { constructor(private repository: IMedicalAppointmentRepository) {} async execute(data: ScheduleRequestDTO): Promise { const appointment = new MedicalAppointment(data); await this.repository.create(appointment); } }
```

```
src/application/use-cases/list-appointments-use-case.ts export class ListAppointmentsUseCase { constructor(private repository: IMedicalAppointmentRepository) {} async execute() { return await this.repository.listAll(); } } ### 3. Camada de Adaptadores
```

(Adapters) O **Controller** recebe os dados da requisição (ex: HTTP) e os repassa para o caso de uso seguindo o padrão de **DTO (Data Transfer Object)** 10, 14. typescript //

```
src/adapters/controllers/medical-appointment-controller.ts import { ScheduleAppointmentUseCase } from "../../application/use-cases/schedule-appointment-use-case"; import { ListAppointmentsUseCase } from "../../application/use-cases/list-appointments-use-case"; export class MedicalAppointmentController { constructor( private scheduleUseCase: ScheduleAppointmentUseCase, private listUseCase: ListAppointmentsUseCase ) {} async handleSchedule(req: any, res: any) { try { const { animalId, description, type, date } =
```

```
req.body; await this.scheduleUseCase.execute({ animalId, description, type, date: new Date(date) }); return res.status(201).send(); } catch (error: any) { return res.status(400).json({ error: error.message }); } } async handleListAll(req: any, res: any) { const appointments = await this.listUseCase.execute(); return res.status(200).json(appointments); } } ### 4.
```

Camada de Infraestrutura (Infrastructure) Aqui reside a implementação concreta do repositório. O uso de um repositório em memória permite testar a lógica antes de conectar um banco de dados real 15-17. typescript //

```
src/infrastructure/repositories/in-memory-appointment-repository.ts import {
IMedicalAppointmentRepository } from
"../../application/repositories/IMedicalAppointmentRepository"; import { MedicalAppointment }
from "../../domain/entities/medical-appointment"; export class
InMemoryMedicalAppointmentRepository implements IMedicalAppointmentRepository {
private appointments: MedicalAppointment[] = []; async create(appointment:
MedicalAppointment): Promise { this.appointments.push(appointment); } async
findAllByAnimal(animalId: string): Promise { return this.appointments.filter(app =>
app.animalId === animalId); } async listAll(): Promise { return this.appointments; } } ###
```

Relação entre Entidades Nesta arquitetura, a relação é baseada no animalId. Ao agendar um exame ou vacina, o sistema assume que o animalId fornecido já existe no seu sistema de cadastro prévio 4, 18. Isso mantém o módulo de saúde (agendamentos) desacoplado do módulo de registro (animais), permitindo que evoluam de forma independente 19, 20.